

Laya Agentic Triage

System-1 / System-2 Agentic Decision Workflow

Project: `laya-agentic-triage`

Version: 0.1.0

Package management: `uv`

Orchestration: LangGraph / LangChain

System-1 model: `convaiinnovations/laya-typed-decisions`

System-2 providers: Groq or Ollama

API: FastAPI

Documentation date: 29 September 2026

This document describes the current V1 implementation and its actual execution behavior. Proposed production upgrades are explicitly separated from current behavior so that routing, scoring, and ownership of decisions remain clear.

System-1

Laya Typed-Decisions

System-2

Groq / Ollama

Orchestration

LangGraph

API

FastAPI

PURPOSE

The project implements a two-speed decision architecture for enterprise incident and support triage. Laya performs fast typed decisions first. LangGraph evaluates risk and confidence and then chooses one of three paths: deterministic automation, System-2 LLM reasoning, or human review. The LLM is therefore an exception-processing and reasoning layer rather than the default classifier.

Core principle: use the smallest specialized model that can make a reliable decision, and invoke a larger generative model only when the decision needs interpretation, explanation, or deeper reasoning.

1. Repository Structure

The repository uses a `src/` layout and keeps application code isolated inside the package namespace.

```
laya-agentic-triage/
├─ pyproject.toml
├─ documentation.md      # This architecture and workflow guide
├─ src/
│   └─ laya_agentic_triage/
│       ├── __init__.py    # Package entrypoint
│       ├── api.py         # FastAPI routes
│       ├── config.py      # Settings and environment
│       ├── graph.py       # LangGraph state machine definition
│       ├── laya_engine.py  # System-1 Laya integration
│       ├── llm.py         # System-2 Groq/Ollama integration
│       ├── main.py        # FastAPI application setup
│       ├── prompts.py     # LLM prompt templates
│       ├── schemas.py     # Pydantic data models
│       ├── state.py       # LangGraph TypedDict state
│       └─ scripts/
│           ├── __init__.py
│           └─ test_graph.py # Evaluation script
└─ README.md
```

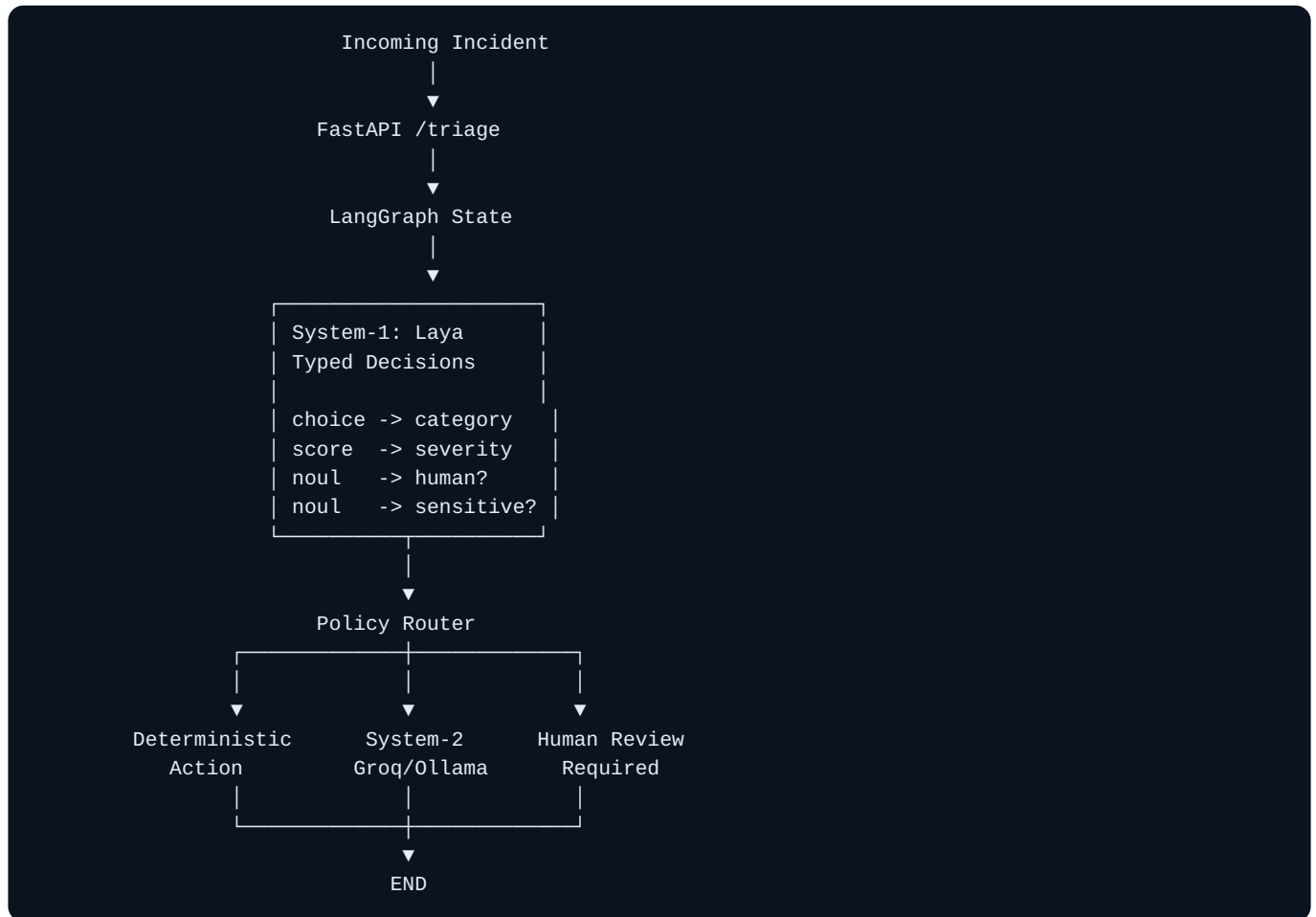
RESPONSIBILITY BOUNDARIES

Module	Responsibility	AI involved?
api.py	HTTP boundary, request/response handling	No
config.py	Environment-driven settings and thresholds	No
graph.py	Workflow orchestration and routing policy	Indirectly
laya_engine.py	Model loading, typed questions, System-1 inference	Laya
llm.py	Provider abstraction for Groq/Ollama	LLM
prompts.py	System-2 reasoning contract	LLM prompt
schemas.py	Typed request/classification/response contracts	No
state.py	Shared LangGraph execution state	No
main.py	FastAPI startup and Laya warm-up	No
scripts/test_graph.py	Local scenario evaluation	Laya + optional LLM

This separation keeps the workflow auditable: model inference, business policy, API transport, and orchestration are not hidden inside a single prompt or function.

2. Architectural Model

The workflow combines three distinct layers.



SYSTEM-1: FAST DECISION LAYER

Laya is not being used as a chatbot. It receives the normalized incident plus a set of typed questions and returns structured decision values and probabilities. In this architecture it is responsible for:

- incident category selection;
- severity scoring;
- probability that human escalation is needed;
- probability that sensitive information is present;
- confidence information used by the router.

SYSTEM-2: REASONING LAYER

Groq or Ollama hosts a generative LLM. It is invoked only for ambiguous low-confidence cases. It receives both the original incident and the System-1 classification and produces an operational explanation and recommended action.

LANGGRAPH: CONTROL PLANE

LangGraph does not classify the ticket itself. It controls state transitions. The routing function implements business policy around confidence, sensitivity, severity, and escalation risk.

3. End-to-End Request Lifecycle

Consider this request:

```
{
  "ticket_id": "INC-101",
  "subject": "Unknown admin login attempts",
  "message": "We detected multiple unknown login attempts and think our API credentials may have leaked.",
  "customer_tier": "enterprise"
}
```

The request flows through the system in the following order.

STEP 1 - FASTAPI RECEIVES THE REQUEST

`POST /trriage` validates the body against `IncidentRequest`. Invalid payloads are rejected before the graph runs. A valid request is passed into the compiled LangGraph as the initial state.

STEP 2 - STATE IS INITIALIZED

Conceptually, the graph begins with:

```
{
  "request": IncidentRequest(...)
}
```

Other fields are populated by nodes as execution proceeds. Nodes do not need to recreate the entire object; they return partial state updates.

STEP 3 - INPUT NORMALIZATION

`normalize_input` trims the subject, collapses message whitespace, and converts request metadata into a predictable dictionary. No model is called here.

STEP 4 - LAYA CLASSIFIES AND SCORES

`laya_classification` sends the normalized incident and all configured typed questions to the resident Laya model. Laya returns a structured answer per question.

STEP 5 - LANGGRAPH APPLIES POLICY

`route_after_laya` evaluates risk before confidence. Critical, human-escalation, or sensitive-data cases go directly to human review. Safe and confident cases take a deterministic action. Remaining ambiguous cases invoke the LLM.

STEP 6 - SELECTED BRANCH EXECUTES

Exactly one branch runs:

- `deterministic_action`, or
- `llm_reasoning`, or
- `human_review`.

Execution then terminates at `END`.

4. Laya Typed Decisions

The V1 workflow asks Laya four independent questions in a single System-1 evaluation call.

```
1. incident_type    -> choice
2. severity         -> score
3. requires_human   -> noul
4. sensitive_data   -> noul
```

The typed questions are business semantics. The probabilities are model outputs.

4.1 choice: incident category

Configured choices include:

- security_incident
- account_access
- billing
- technical_support
- service_request
- other

For a security-related message, a conceptual result could be:

```
{
  "choice": "security_incident",
  "probabilities": {
    "security_incident": 0.91,
    "account_access": 0.04,
    "billing": 0.01,
    "technical_support": 0.02,
    "service_request": 0.01,
    "other": 0.01
  },
  "answer_confidence": 0.91
}
```

The application does **not** manually assign `0.91`. Laya calculates a probability distribution from its model outputs and returns the selected option.

4.2 Why criteria descriptions matter

Each label is paired with a semantic description. These criteria define what each class means in the domain and reduce ambiguity between adjacent categories such as `account_access` and `technical_support`.

The criteria should be treated as part of the model contract and versioned alongside evaluation data.

5. Severity Score: How It Is Produced

Severity uses Laya's ordered `score` primitive rather than a free-form numeric prediction.

The configured ordered levels are:

```
0 = low
1 = medium
2 = high
3 = critical
```

Suppose Laya produces this distribution:

```
{
  "0": 0.02,
  "1": 0.08,
  "2": 0.72,
  "3": 0.18
}
```

Laya computes the expected position across the ordered levels:

```
score = (0 x 0.02)
        + (1 x 0.08)
        + (2 x 0.72)
        + (3 x 0.18)
        = 2.06
```

The application then maps the continuous score to a readable label:

```
index = round(score)
label = SEVERITY_LABELS[index]
```

For `2.06` :

```
round(2.06) = 2
2 = high
```

IMPORTANT DISTINCTION

`severity_score` and `severity_confidence` are not the same concept.

```
severity_score      = position on the low -> critical scale
severity_confidence = confidence in the typed severity answer
```

A ticket could have a severity score near `2.0` while the model is uncertain whether it is `medium` or `high`. That uncertainty is why routing should use confidence separately from the score itself.

6. noul: Binary Probability Decisions

The workflow uses the `noul` primitive for two binary questions:

```
Does this require human escalation?  
Does this contain or indicate sensitive data?
```

Laya returns `noul` as **P(true)**.

Example:

```
{  
  "requires_human": {  
    "noul": 0.88  
  },  
  "sensitive_data": {  
    "noul": 0.76  
  }  
}
```

Interpretation:

```
P(human escalation required) = 0.88  
P(sensitive data present)    = 0.76
```

These are model probabilities. The application controls what probability becomes actionable through policy thresholds.

MODEL OUTPUT VS. POLICY

LAYA OUTPUT	APPLICATION POLICY
-----	-----
0.88 human probability ->	compare against 0.65
0.76 sensitive probability ->	compare against 0.65

The threshold is deliberately external to the model so it can be tuned through evaluation without retraining Laya.

7. Confidence and Business Thresholds

The application currently defines:

```
LAYA_CONFIDENCE_THRESHOLD = 0.80
LAYA_HUMAN_THRESHOLD      = 0.65
```

These values are **V1 business defaults**, not universal Laya recommendations. They must be calibrated against the project's own incident dataset before production use.

7.1 answer_confidence

The workflow gates on Laya's `answer_confidence` field. Laya exposes this as the calibrated confidence quantity that can be used consistently across typed question types.

For the current V1, only category and severity confidence are combined:

```
overall_confidence = min(
    category_confidence,
    severity_confidence,
)
```

Example:

```
category confidence = 0.92
severity confidence = 0.84

V1 overall confidence = min(0.92, 0.84)
                      = 0.84
```

WHY MIN()?

The design intentionally uses the weakest important decision rather than an average.

```
category = 0.95
severity = 0.52
```

An average could obscure severity uncertainty. `min()` keeps routing conservative:

```
overall = 0.52
```

This is an application design choice, not a Laya requirement.

8. Exact LangGraph Routing Policy

The router prioritizes **risk before automation confidence**.

```
graph TD
    START --> normalize_input
    normalize_input --> laya_classification
    laya_classification --> route_after_laya
    route_after_laya -- "severity == critical ? ————— yes -->" --> human_review
    route_after_laya -- "human_probability >= 0.65 ? ——— yes -->" --> human_review
    route_after_laya -- "sensitive_probability >= 0.65 ? – yes -->" --> human_review
    route_after_laya -- "overall_confidence >= 0.80 ? ——— yes -->" --> deterministic_action
    route_after_laya -- "otherwise —————->" --> llm_reasoning
```

Equivalent policy pseudocode:

```
if result.severity == "critical":
    return "human_review"

if result.human_escalation_probability >= HUMAN_THRESHOLD:
    return "human_review"

if result.sensitive_data_probability >= HUMAN_THRESHOLD:
    return "human_review"

if result.overall_confidence >= CONFIDENCE_THRESHOLD:
    return "deterministic"

return "llm_reasoning"
```

WHY RISK CHECKS COME FIRST

A high-confidence prediction does not imply that automation is appropriate. A ticket can be confidently recognized as a critical security incident and still require a human response.

High confidence ≠ safe to automate

That is the most important policy property in the graph.

9. Scenario A - High Confidence, Low Risk

EXAMPLE INPUT

```
Subject: Change timezone
Message: How do I change the timezone in my account settings?
```

ILLUSTRATIVE LAYA OUTPUT

```
incident_type      service_request
category_confidence 0.94
severity           low
severity_confidence 0.91
human_probability  0.05
sensitive_probability 0.01
overall_confidence 0.91
```

ROUTE EVALUATION

```
critical?          no
human >= 0.65?     no
sensitive >= 0.65? no
overall confidence >= 0.80? yes
```

EXECUTED GRAPH

```
START
↓
normalize_input
↓
Laya
↓
deterministic_action
↓
END
```

The queue mapping sends `service_request` to `Customer Success`.

LLM PARTICIPATION

None. Groq/Ollama is never called.

This path provides the main latency and cost advantage of the architecture: ordinary well-understood requests do not pay generative-model inference cost.

10. Scenario B - Ambiguous Classification

EXAMPLE INPUT

Our integration stopped working after the latest account changes.
We cannot determine whether this is a permission problem or an API problem.

ILLUSTRATIVE LAYA OUTPUT

```
technical_support 0.47
account_access    0.43
other             0.10

severity confidence = 0.69
overall confidence  = 0.47
human probability   = 0.34
sensitive probability = 0.08
```

ROUTE EVALUATION

The request is not critical and does not cross the human/sensitive thresholds, but confidence is below `0.80`.

```
0.47 < 0.80
```

LangGraph therefore routes to `llm_reasoning`.

WHAT SYSTEM-2 RECEIVES

The LLM receives two evidence blocks:

1. Original normalized incident
2. Laya System-1 classification

The prompt explicitly tells the LLM not to blindly accept Laya's result. It may state that the likely cause is account access rather than technical support and explain why.

EXECUTED GRAPH

```
START -> normalize -> Laya -> low confidence -> Groq/Ollama -> END
```

11. Scenario C - Critical or Sensitive Incident

EXAMPLE INPUT

```
We believe our production API private key was exposed and unknown logins are appearing.
```

ILLUSTRATIVE LAYA OUTPUT

```
category          security_incident
category confidence 0.97
severity          critical
human probability   0.96
sensitive probability 0.91
overall confidence  0.94
```

This is a useful example because the model is highly confident. However, the risk policy wins before confidence is considered.

ROUTE EVALUATION

```
severity == critical -> TRUE
```

The graph immediately chooses `human_review`.

EXECUTED GRAPH

```
START
↓
normalize
↓
Laya
↓
critical / sensitive
↓
human_review
↓
END
```

LLM PARTICIPATION

None in V1. The system does not ask a generative model to decide whether a critical credential exposure should be automatically handled.

This route is intentionally conservative and keeps high-risk decisions explicit and auditable.

12. Scenario D - High Confidence but Human Escalation Probability Crosses Threshold

A ticket can be non-critical yet still require review.

Example:

```
Category confidence    0.93
Severity              high
Severity confidence    0.87
Human probability      0.72
Sensitive probability  0.20
```

Although `overall_confidence >= 0.80`, the human probability is evaluated first:

```
0.72 >= 0.65 -> human_review
```

WHY THIS EXISTS

Confidence answers the question:

How strongly does the model support its typed answer?

Human escalation probability answers a different question:

Does the nature of this case warrant a person in the loop?

Conflating the two would make the system unsafe. High-confidence cases may still involve refunds, enterprise-impacting failures, legal questions, unusual account actions, or policy exceptions that require a person.

13. Scenario E - Sensitive Data Threshold Only

Consider a request that looks operationally simple but contains sensitive material.

```
I pasted the production token below because the integration returns 401.
```

Possible output:

```
incident_type      technical_support
category confidence 0.90
severity           medium
severity confidence 0.86
human probability  0.44
sensitive probability 0.82
```

Route:

```
sensitive_probability >= 0.65
-> human_review
```

The deterministic confidence path is never evaluated after this condition succeeds.

PRODUCTION IMPLICATION

The next production version should add a pre-LLM redaction or secret-detection layer so sensitive content is not forwarded to remote model providers unless policy permits it. V1 currently routes these cases away from the LLM, which already reduces exposure.

14. Scenario F - Threshold Boundary Behavior

Threshold equality matters because the code uses `>=`.

```
human probability = 0.6500
human threshold   = 0.6500
```

Result:

```
0.6500 >= 0.6500 -> human_review
```

Likewise:

```
overall confidence = 0.8000
confidence threshold = 0.8000
```

If no risk condition was triggered:

```
0.8000 >= 0.8000 -> deterministic_action
```

WHY DOCUMENT BOUNDARIES

Evaluation tests should contain exact threshold cases. Otherwise a future refactor from `>=` to `>` can silently change production routing around the boundary.

Recommended tests:

```
0.6499 / 0.6500 / 0.6501
0.7999 / 0.8000 / 0.8001
```

15. System-1 and System-2 Participation

The architecture deliberately separates **decision primitives** from **generative reasoning**.

Capability	Laya	Groq/Ollama LLM
Category classification	Primary	Can reconsider
Ordered severity scoring	Primary	Can discuss/reconsider
Binary risk decisions	Primary	Can reason about implications
Confidence probabilities	Primary	Not used as authoritative classifier confidence
Natural-language explanation	No	Primary
Operational recommendation	Limited/No	Primary
Long-form response	No	Primary
Tool calling	No in this workflow	Future extension
Deterministic routing	No	No - LangGraph policy

CORE COST/LATENCY PATTERN

Easy case

-> Laya only

Ambiguous case

-> Laya + LLM

Risk case

-> Laya + human-review route

The LLM is therefore not a more expensive replacement for the classifier. It is a second reasoning stage used only when the classifier is insufficient for safe automatic action.

16. What the LLM Does After Laya

`llm_reasoning` is invoked only on the ambiguous branch.

The prompt supplies:

```
Original incident
+
System-1 structured classification
```

The System-2 contract asks for:

```
Summary
Category
Severity
Reasoning
Recommended team
Recommended action
Human review required
```

The LLM can explicitly disagree with Laya. This is important because low-confidence routing exists precisely for cases where System-1 evidence is weak or split across nearby categories.

CURRENT V1 LIMITATION

The LLM response is plain text and becomes `final_response`.

It does **not** overwrite:

```
state["classification"]
```

Therefore the API may expose:

```
classification.incident_type = technical_support
```

while the textual LLM analysis says:

```
Likely category: account_access
```

This is intentional in V1 because it preserves the original System-1 output for debugging, but it means the LLM is advisory rather than authoritative.

17. State Evolution Through the Graph

LangGraph's shared state makes execution inspectable.

INITIAL STATE

```
{
  "request": IncidentRequest(...)
}
```

AFTER NORMALIZE_INPUT

```
{
  "request": ...,
  "normalized_input": {
    "ticket_id": "INC-101",
    "subject": "...",
    "message": "...",
    "customer_tier": "enterprise",
    "metadata": {}
  }
}
```

AFTER LAYA_CLASSIFICATION

```
{
  ...,
  "classification": ClassificationResult(...)
}
```

AFTER DETERMINISTIC PATH

```
{
  ...,
  "route": "deterministic",
  "final_response": "Automatically routed to ..."
}
```

AFTER LLM PATH

```
{
  ...,
  "route": "llm_reasoning",
  "llm_analysis": "...",
  "final_response": "..."
}
```

The state object therefore doubles as an execution record that can later be checkpointed, traced, or persisted.

18. Singleton Model Loading and Concurrency

`get_laya_agent()` uses `@lru_cache(maxsize=1)`.

Within one Python process:

```
first request -> load model
later requests -> reuse same resident model
```

`main.py` warms the singleton during FastAPI lifespan startup, avoiding first-request model initialization latency.

IMPORTANT PROCESS BOUNDARY

`uvicorn --workers 1` is recommended for V1 because each process has independent Python memory.

```
worker 1 -> Laya copy 1
worker 2 -> Laya copy 2
worker 3 -> Laya copy 3
```

`lru_cache` is process-local, not cluster-global.

SEMAPHORE

The implementation also wraps inference with:

```
BoundedSemaphore(LAYA_MAX_CONCURRENCY)
```

Starting with `1` keeps GPU memory behavior predictable during local testing. This does not increase model copies; it limits simultaneous calls into the shared resident model.

Production concurrency should be load-tested rather than increased blindly.

19. FastAPI Execution and Event-Loop Protection

The graph currently contains synchronous PyTorch and LangChain calls. Running them directly inside an async FastAPI handler would block the event loop.

The endpoint therefore uses:

```
await asyncio.to_thread(
    triage_graph.invoke,
    {"request": request},
)
```

This lets the synchronous graph run in a worker thread while the server event loop remains available for other HTTP activity.

WHAT THIS DOES NOT SOLVE

`asyncio.to_thread` does not make GPU inference itself asynchronous and does not remove the semaphore. It only prevents a synchronous graph invocation from occupying the FastAPI event-loop thread.

For higher throughput later, batching at the Laya layer can be more useful than simply increasing request-level concurrency because Laya exposes batch-oriented inference APIs.

20. Groq and Ollama Provider Switching

The System-2 model is abstracted behind `get_llm()`.

```
LLM_PROVIDER=groq
|
└-> ChatGroq(...)

LLM_PROVIDER=ollama
|
└-> ChatOllama(...)
```

The graph itself does not change when the provider changes.

GROQ MODE

Suitable when low-latency hosted inference is acceptable and network/provider policy permits sending the ambiguous incident context externally.

OLLAMA MODE

Suitable when the selected model should run locally or inside controlled infrastructure.

ARCHITECTURAL VALUE

Provider switching is a System-2 infrastructure concern rather than a graph concern. This preserves the workflow while allowing model experimentation.

A production implementation should additionally record provider/model/version in trace metadata so evaluations can distinguish routing quality from System-2 model behavior.

21. Failure and Edge Scenarios

V1 catches unhandled endpoint exceptions and returns HTTP 500. Production behavior should be more granular.

Invalid request schema

Pydantic/FastAPI rejects malformed bodies before graph execution.

Laya model load failure

Startup warm-up should fail deployment readiness rather than accepting traffic with no System-1 classifier.

Laya inference failure

A production policy should decide whether to:

- fail closed into human review;
- use a temporary LLM fallback classifier;
- return a retrievable service error.

For safety-sensitive incident triage, fail-closed human review is usually easier to audit.

Groq network/provider failure

Only ambiguous requests depend on System-2. Deterministic and human-review routes should continue functioning if Groq is unavailable.

Ollama unavailable

The ambiguous branch should return a controlled fallback rather than crash the complete API.

Unknown category mapping

`QUEUE_MAP.get(..., "General Support")` provides a deterministic default queue.

Oversized input

The typed-decisions checkpoint has a finite model context. Long ticket histories or attachments need explicit long-input handling rather than assuming the entire state was evaluated.

22. Long Inputs and Context Windows

The Laya typed-decisions checkpoint is based on ModernBERT-large and is configured with a finite input length. Current Laya runtime code also exposes `predict_long` for sliding-window processing of long states.

This matters for future tickets that contain:

- complete email threads;
- long incident timelines;
- log excerpts;
- attachment text;
- multi-message customer conversations.

CURRENT V1

The project calls the normal prediction path on one normalized request.

PRODUCTION OPTION

For long content, use an explicit strategy:

- A. pre-summarize evidence safely
- B. select relevant sections before System-1
- C. use Laya `predict_long` with window metadata
- D. split ticket body from attachments and classify separately

When window aggregation is used, document which window produced the deciding answer. Do not treat a deciding-window confidence as automatically calibrated for an entire large document.

23. Evaluation Before Production

Do not tune thresholds by intuition alone.

Create a labeled validation set containing realistic examples for every route and category.

EVALUATE SYSTEM-1

Measure:

- category accuracy;
- confusion matrix;
- severity MAE / ordinal error;
- human escalation precision and recall;
- sensitive-data recall;
- calibration / ECE;
- routing rate by confidence bucket.

EVALUATE GRAPH BEHAVIOR

Measure:

```
% deterministic
% LLM fallback
% human review
false automation rate
unnecessary human-review rate
LLM disagreement rate
```

THRESHOLD TUNING OBJECTIVE

The optimal threshold is domain-dependent. For security/sensitive-data gating, recall may matter more than automation rate. For harmless service requests, higher deterministic coverage may be acceptable.

Thresholds should be versioned and changed through controlled evaluation.

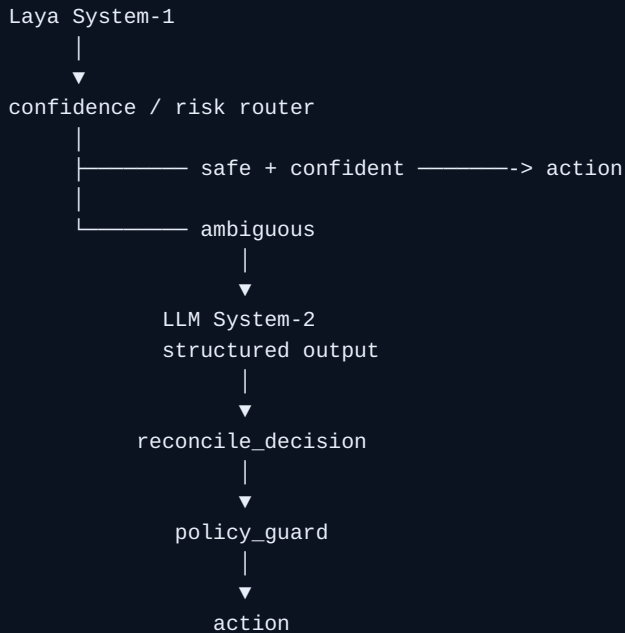
24. Recommended Scenario Test Matrix

ID	Situation	Expected route	LLM called?
T01	Simple service request, high confidence	deterministic	No
T02	Billing request, high confidence, low risk	deterministic	No
T03	Ambiguous account vs API issue	llm_reasoning	Yes
T04	Critical security incident	human_review	No
T05	Sensitive credential text	human_review	No
T06	Human probability exactly 0.65	human_review	No
T07	Confidence exactly 0.80, low risk	deterministic	No
T08	Confidence 0.7999, low risk	llm_reasoning	Yes
T09	High confidence + human probability 0.72	human_review	No
T10	High confidence + sensitive probability 0.82	human_review	No
T11	Unknown category key	deterministic default queue	No
T12	Groq unavailable on ambiguous case	controlled System-2 failure/fallback	Attempted
T13	Laya unavailable	fail closed / readiness failure	No
T14	Long input	explicit long-input policy	Depends

This table should become automated regression coverage rather than only manual testing.

25. Production Evolution: Decision Reconciliation

The most important V2 improvement is structured System-2 output plus explicit reconciliation.



The LLM should return a Pydantic object such as:

```
recommended_category
recommended_severity
human_review_required
reasoning_summary
confidence_or_evidence_notes
```

The reconciliation node can then preserve both:

```
system_1_decision
system_2_decision
final_decision
```

This is preferable to silently overwriting Laya because disagreements become observable and measurable.

26. Production Evolution: Guardrails, Persistence, and Observability

After the core routing is validated, add infrastructure in this order.

1. STRUCTURED SYSTEM-2 OUTPUT

Eliminate free-form parsing and formalize reconciliation.

2. POLICY GUARD

Apply final rules after System-2 so a generative model cannot bypass sensitive/critical policies.

3. LANGGRAPH CHECKPOINTING

Persist graph state for retries, long-running human review, and auditability.

4. LANGFUSE OR EQUIVALENT TRACING

Trace:

```
request id
Laya model/version
Laya probabilities
thresholds
selected route
LLM provider/model
latency per node
final disposition
human override
```

5. SECRET/PII PROTECTION

Redact or tokenize sensitive fields before remote LLM calls and define clear provider boundaries.

6. QUEUE / BACKPRESSURE

Introduce asynchronous work queues only after observed load requires them.

7. BATCH SYSTEM-1 INFERENCE

Use batching to improve throughput under sustained traffic.

27. Security and Data Handling

This project is an incident-triage system, so model routing is also a data-routing decision.

CURRENT SAFETY ADVANTAGE

Sensitive incidents are routed away from the LLM in V1 when Laya's sensitive probability crosses the configured threshold.

ADDITIONAL CONTROLS RECOMMENDED

- never log raw secrets;
- redact credentials before traces;
- avoid placing access tokens in prompt metadata;
- define whether Groq is permitted for confidential incident text;
- use Ollama/local infrastructure where policy requires local processing;
- encrypt persisted graph state;
- use request IDs rather than unnecessary personal identifiers in observability;
- record human overrides without storing avoidable sensitive payloads;
- separate operational logs from model-evaluation datasets.

FAIL-CLOSED PRINCIPLE

If the system cannot confidently determine that a request is safe for automatic processing, route toward review or controlled fallback rather than expanding automation implicitly.

28. Local Development with uv

The project is designed around `uv` for environment and dependency management.

Typical setup:

```
uv init --python 3.12
uv add langchain langgraph langchain-groq langchain-ollama \
  "laya[langgraph]" fastapi "uvicorn[standard]" \
  pydantic pydantic-settings python-dotenv
```

Run the evaluation script using the `src` layout:

```
uv run python -m laya_agentic_triage.scripts.test_graph
```

Run FastAPI:

```
uv run uvicorn laya_agentic_triage.main:app \
  --app-dir src \
  --host 0.0.0.0 \
  --port 8000 \
  --workers 1
```

WHY ONE WORKER INITIALLY

Each worker is a separate process and would load a separate model copy. Start with one resident Laya instance, validate memory and latency, and scale using measured deployment behavior rather than assuming multi-worker replication is free.

29. Operational Mental Model

The complete architecture can be summarized as five questions.

1. WHAT IS THIS REQUEST?

Laya `choice` determines the best incident category.

2. HOW SEVERE IS IT?

Laya `score` places the incident on an ordered low-to-critical scale.

3. IS IT RISKY TO AUTOMATE?

Laya `noul` estimates human-escalation and sensitive-data probabilities.

4. IS SYSTEM-1 SUFFICIENTLY CERTAIN?

LangGraph compares calibrated decision confidence with the configured automation threshold.

5. WHAT SHOULD HAPPEN NEXT?

LangGraph selects deterministic action, System-2 reasoning, or human review.

```
Laya answers typed questions.  
LangGraph applies policy.  
LLM reasons about exceptions.  
Humans retain high-risk control.
```

That separation is the central architectural idea of the project.

30. Current V1 vs. Target Production Architecture

Area	Current V1	Production target
System-1	Laya singleton	Laya + calibrated domain evaluations
System-2	Free-form LLM analysis	Structured Pydantic decision
Reconciliation	None	Explicit S1/S2 reconciliation node
Policy	Pre-LLM router	Pre- and post-LLM policy guards
State	In-memory invocation	Durable checkpointing
Human review	Route/message only	Pausable HITL workflow
Observability	Console/basic API	Langfuse traces + metrics
Sensitive data	Laya gating	Gating + redaction + provider policy
Failure handling	HTTP 500	Typed retry/fail-closed policies
Throughput	Semaphore, one worker	Load-tested batching/backpressure
Evaluation	Script scenarios	Regression dataset + route metrics

The recommendation is to validate classification quality and routing behavior before adding infrastructure. Complexity should be introduced only where evaluation or load measurements justify it.

31. References and Implementation Notes

LAYA

- Model: `convaiinnovations/laya-typed-decisions`
- Model card: <https://huggingface.co/convaiinnovations/laya-typed-decisions>
- Runtime source: <https://github.com/NandhaKishorM/laya/blob/main/laya/agent.py>

The current model card describes the typed-decisions checkpoint as a 421M-parameter ModernBERT-large model with a 1024-token configured context, specialized for agent-trace observability, customer service, invoice processing, and security incidents.

The current Laya runtime source documents:

- `choice` returning a selected option and probabilities;
- `score` returning the expected ordered position;
- `noul` returning $P(\text{true})$;
- `answer_confidence` as the calibrated confidence field across question types;
- `predict_batch` for shared forward passes;
- `predict_long` for long-state windowing and aggregation.

LANGGRAPH

- Graph API: <https://docs.langchain.com/oss/python/langgraph/graph-api>
- StateGraph reference: <https://reference.langchain.com/python/langgraph/graph/state/StateGraph>

LangGraph provides explicit state, nodes, edges, and conditional routing. In this project it is the control plane rather than the classifier.

DESIGN RULE

Model outputs and application policy must remain separate:

```
Model: probability / typed answer
Policy: threshold / route / action
```

Keeping this boundary explicit makes the system testable, calibratable, and easier to audit.